

UNIVERSITÀ
DI TRENTO

Department of Information Engineering and Computer Science

Bachelor's Degree in Computer, Communications and Electronic Engineering

Final Dissertation

Evolving Neuron-Centric Hebbian Learning for Soft-Robot Control

Supervisor
Giovanni Iacca

Student
Rowan Li Tangalin

Academic year 2026/2027

Acknowledgements

To be completed.

Contents

Use of Artificial Intelligence Tools **3**

Abstract **3**

1 Introduction **3**
1.1 Research Questions and Contributions 4
1.2 Thesis Outline 4

2 Background **4**
2.1 EvoGym 4
2.1.1 Morphologies 5
2.1.2 Tasks 5
2.2 Hebbian Learning 6
2.2.1 ABCD Hebbian Learning Rules 6
2.2.2 Neuron-centric Hebbian Learning 6
2.3 Evolution 6
2.3.1 Genetic Algorithms 6
2.3.2 ES1 7

3 Original Study: Methodology and Results **7**
3.1 Shared Experimental Framework 7
3.1.1 Implementation and Reproducibility 7
3.1.2 Controller Implementations 7
3.1.3 Tasks, Morphologies, and Evolution 8
3.2 RQ1: Evolutionary Training Dynamics 9
3.2.1 Experimental Design 9
3.2.2 Results 9
3.3 RQ2: Robustness to Unseen Initial Weights 11
3.3.1 Experimental Design 11
3.3.2 Results 11
3.4 RQ3: Do Lifetime Updates Create Useful Weights? 12
3.4.1 Freeze-and-Replay Design 12
3.4.2 Results 12

4 Extended Study: Task and Morphology Dependence **14**
4.1 Motivation and Research Question 14
4.2 Experimental Design 14

4.2.1 Task Selection and Predefined Groups . . . 14
4.2.2 Control Demands of the Individual Tasks . . . 14
4.2.3 Controlled Morphologies . . . 14
4.2.4 Experimental Scale and Completeness . . . 15
4.3 Analysis Strategy . . . 15
4.4 Results Across Tasks and Morphologies . . . 15
4.4.1 Training Dynamics and Task-Group Results . . . 15
4.4.2 Task-by-Morphology Results . . . 16
4.5 Task Characteristics and Answer to RQ4 . . . 17
4.5.1 Favourable and Unfavourable Task Demands . . . 17
4.5.2 Morphology Dependence . . . 18
4.5.3 RQ4 Summary . . . 19

5 Conclusion **19**
5.1 Conclusions and Limitations . . . 19
5.2 Future Work . . . 19

Bibliography **19**

Use of Artificial Intelligence Tools

OpenAI Codex was used to support language revision of the thesis, figure preparation, and plotting.

Abstract

Soft robotics refers to the study of robotic systems that exploit deformable bodies in order to interact with complex environments. The same characteristics which enable these capabilities also make the systems more difficult to control. Additionally, certain applications of soft robotics, such as search-and-rescue in unstructured terrain, may benefit from the ability to adapt and learn during the lifetime of the robot in order to face unforeseen changes or simply optimize their behavior.

Biological systems achieve adaptability in part through synaptic plasticity, where the connections between neurons change according to their activity. Hebbian Learning provides a simple model of this principle by modifying neural-network weights online according to the activation of connected neurons.

Conventional Hebbian formulations associate plasticity parameters with individual synapses, causing the number of parameters to grow with the number of connections. Neuron-centric Hebbian Learning (NcHL) addresses this scaling problem by associating the parameters governing plasticity with the neurons instead, substantially reducing the dimensionality of the learning rule while preserving activity-dependent adaptation.

This project investigates whether NcHL can provide competitive and adaptive control for soft robots. In particular, we compare NcHL-based controllers with static neural controllers across different tasks and morphologies, examine the robustness of evolved NcHL rules to unseen neural-network initializations, and investigate whether NcHL produces meaningful learning within an individual lifetime.

The second, larger experiment extends the controller comparison to twelve tasks and three controlled morphologies. Tasks are grouped in advance by dominant control demand—continuous-support locomotion, terrain negotiation, object interaction, and whole-body dynamic control—so that the analysis can test whether the relative performance of NcHL follows an interpretable task-level pattern.

1 Introduction

Soft robots exploit deformable bodies to interact with environments in ways that are difficult for rigid mechanisms. Their compliance can support safe contact and adaptation to irregular terrain, but it also couples sensing, control, and body dynamics tightly [5]. An autonomous soft robot is therefore useful only while the relationship between its sensors, controller, morphology, and environment remains suitable for the task.

However, conventional optimization usually produces controller parameters that remain fixed after training. The controller may generate different actions for different observations, but the mechanism that maps observations to actions cannot itself adapt during deployment. This is a restrictive assumption when the robot encounters changes that were absent from training, and it is especially relevant for soft bodies whose physical response is difficult to model precisely. Evolved plastic neural controllers address this problem by separating two processes: evolution searches across generations for a learning rule, and that rule modifies network weights during the lifetime of an individual [6].

1.1 Research Questions and Contributions

This thesis investigates Neuron-centric Hebbian Learning (NcHL), which reduces the number of evolved plasticity parameters by associating them with neurons rather than individual synapses [3]. It compares an evolved NcHL controller with a static neural controller while holding the morphology, observations, action mapping, network topology, evolutionary optimizer, and task horizon fixed within each experimental condition. The investigation is organized around four research questions.

RQ1 — Evolutionary training dynamics. How does the best-so-far fitness of NcHL-ES develop during evolution relative to Static-ES across the original task–morphology conditions?

RQ2 — Robustness to initialization. Does an evolved NcHL rule remain effective when the concrete neural-network weights are initialized with random seeds not encountered during evolution?

RQ3 — Within-lifetime learning. Do the weight updates produced by NcHL create behaviourally useful weights, as opposed to weight change without a corresponding performance benefit?

RQ4 — Task and morphology dependence. Which task characteristics are associated with good NcHL performance, and are those patterns independent of morphology?

The thesis makes three principal contributions:

1. A controlled comparison of NcHL-ES and Static-ES on the original EvoGym task and morphology suite, including evolutionary training dynamics and qualitative inspection of the evolved behaviour;
2. Two mechanistic evaluations of the learned plasticity rules, testing sensitivity to unseen initial weights and whether weights acquired during a lifetime remain useful when frozen and replayed;
3. An extension to twelve tasks and three controlled morphologies, combining task-group analysis, task–morphology comparisons, and controller footage to investigate whether NcHL performance follows task characteristics or body geometry.

The study validates these contributions with ten independent evolutionary runs per condition, generation-level fitness curves, evaluations across unseen initializations, freeze-and-replay interventions, standardized comparisons across task-specific reward scales, and qualitative footage analysis. The results show that NcHL can produce meaningful locomotion and object-interaction behaviour, and that its online updates often improve on random initial weights. They also show sensitivity to initialization, no consistent task-class advantage, and a substantially clear dependence on morphology.

1.2 Thesis Outline

Chapter 2 introduces EvoGym, Hebbian learning, NcHL, and the evolutionary optimizer used in the experiments. Chapter 3 presents the original study and answers RQ1–RQ3 through its methodology and results. Chapter 4 extends the comparison across tasks and controlled morphologies and answers RQ4. Chapter 5 summarizes the conclusions, limitations, and directions for future work.

2 Background

2.1 EvoGym

Evolution Gym, usually shortened to EvoGym, is a benchmark for the joint optimization of the design and control of two-dimensional soft robots [1]. It provides a common representation for robot bodies and a collection

of environments in which those bodies can be controlled. Its simulator models voxel-based objects using an underlying mass–spring system, advances their physical state over discrete timesteps, and exposes the resulting control problem through a Gym-compatible interface.

An EvoGym interaction follows the usual agent–environment loop. At timestep t, the environment provides an observation $\mathbf{o}_t$. The controller maps it to an action $\mathbf{u}_t$, which specifies a deformation scale for each of the robot's actuators. After simulating the action, the environment returns a new observation, a scalar reward r_t, and an indication of whether the episode has ended. The return of one episode is

$$F = \sum_{t=0}^{T-1} r_t, \tag{2.1}$$

where T is the episode horizon or the timestep at which the environment terminates. This return is used as fitness by the evolutionary optimizer [1].

EvoGym is useful for this project because it separates three objects that need to remain conceptually distinct: morphology, task, and controller. Holding two of these objects fixed facilitates the study of the third.

2.1.1 Morphologies

An EvoGym morphology is a rectangular matrix of integer material codes. Each non-empty matrix cell represents one square voxel. Adjacent occupied cells are connected in the physical simulation, so the matrix specifies both the material distribution and the geometry of the robot. A valid robot body must form one connected structure and must contain at least one actuator [1].

Table 2.1 lists the material codes used by the morphologies in this thesis. Rigid voxels resist deformation and provide structural support. Soft voxels deform passively in response to forces. Horizontal and vertical actuators actively expand or contract along their named axis [1].

Table 2.1: EvoGym voxel codes used to describe robot morphologies.

Code	Type	Role in a morphology
0	Empty	No robot material is placed in the cell.
1	Rigid	Provides an occupied, non-deformable structural element.
2	Soft	Deforms passively under internal and external forces.
3	Horizontal actuator	Changes its preferred width in response to a control signal.
4	Vertical actuator	Changes its preferred height in response to a control signal.

The number of actuators determines the size of the controller's output, while the number and arrangement of voxels affect the observation and the physical consequences of each action.

2.1.2 Tasks

In EvoGym, a task defines the terrain and other simulated objects, the initial placement of the robot, the observations available to the controller, the admissible actuator commands, the reward calculated after each transition, and the conditions that terminate an episode.

Observations contain task-specific information about the robot and the surrounding world. Typical robot information includes velocities and the positions of voxel corner masses relative to the robot's centre of mass. Environments that contain a movable object or a target can add relative positions, object velocities, or other quantities required to make progress observable. The observation is presented to the controller as a numerical vector and the controller is not given a symbolic description of the task.

The action vector contains one command for every actuator voxel in the morphology. A command changes the actuator's target length, causing expansion or contraction. Useful motion must emerge from the timing of these local deformations, the passive response of the body, and contact with the environment. Rigid and passive soft voxels do not receive commands.

Reward gives numerical feedback about progress. Its definition is part of the task: it may value displacement, object motion, height, reaching a target, or a combination of several terms. Consequently, return is meaningful

within a task but is not a universal unit of performance. A return of a particular magnitude in one EvoGym environment cannot be interpreted as better than the same number in another environment without an explicit normalization [1].

2.2 Hebbian Learning

Hebbian learning is a concept built around manipulating synaptic weight based on the activation of the two neurons connected by that synapse. Unlike supervised backpropagation, a Hebbian update does not require a target output or the gradient of a global loss. The central idea is to use a learning rule to produce useful updates to the weights of the network based on its own current state and observations [3, 4].

2.2.1 ABCD Hebbian Learning Rules

A generalized ABCD rule represents four basic influences on a connection from presynaptic neuron i to postsynaptic neuron j [3, 4]:

$$\Delta w_{ij} = \eta \left(A_{ij} a_i + B_{ij} a_j + C_{ij} a_i a_j + D_{ij} \right), \tag{2.2}$$

where a_i and a_j are the current neuron activations and η is a learning-rate scale. The A and B terms depend on the presynaptic and postsynaptic activity, respectively. The C term can reinforce or suppress a connection according to joint activity, while D allows activation-independent changes.

2.2.2 Neuron-centric Hebbian Learning

Neuron-centric Hebbian Learning moves the evolved parameters from synapses to neurons [3]. In the published formulation, each neuron k stores one tuple

$$\boldsymbol{\theta}_k = (A_k, B_k, C_k, D_k, \eta_k). \tag{2.3}$$

The update for a connection is assembled from the tuples belonging to its presynaptic and postsynaptic endpoints. The rule uses the mean of their learning rates:

$$\Delta w_{ij} = \frac{\eta_i + \eta_j}{2} \left(A_i a_i + B_j a_j + C_i C_j a_i a_j + D_i D_j \right). \tag{2.4}$$

2.3 Evolution

Evolutionary computation searches over a population of candidate solutions using fitness evaluation, selection, and variation, without requiring gradient information [2]. This is useful in EvoGym because fitness depends on a simulated trajectory containing contacts, deformation, and many successive controller decisions. It is also useful for plastic controllers: evolution can optimize the coefficients of a learning rule using only the fitness of the individual.

In the terminology used here, a *genome* is the numerical vector inherited and modified by the optimizer, an *individual* is one candidate genome, and a *population* is the set of candidates evaluated in a generation. The meaning of a gene depends on the controller representation. A static genome stores connection weights. An NcHL genome stores neuron-level learning-rule coefficients. In both cases, EvoGym episode return supplies the fitness used to rank candidates.

2.3.1 Genetic Algorithms

A Genetic Algorithm (GA) is a population-based evolutionary algorithm built around repeated evaluation, selection, and variation. An initial population is generated, every individual is assigned a fitness, and better-performing individuals are given a greater role in producing the next generation. Variation operators such as mutation and crossover create new genomes from the selected parents. The cycle continues until an evaluation budget or another stopping condition is reached.

Selection makes the search exploit information already discovered, while variation explores nearby or recombined solutions. Mutation changes genes stochastically; crossover combines material from multiple parents. Elitism may copy one or more strong individuals unchanged so that an already discovered solution is not lost through random variation.

These components describe a large family of mathematical algorithms which can work in discrete or continuous domains. Specific evolutionary optimizers choose different representations, selection rules, and variation distributions [2].

2.3.2 ES1

ES1 is the chosen strategy; it uses the arithmetic mean of the best μ genomes. If $\mathbf{g}_1, \ldots, \mathbf{g}_\mu$ are the selected parents, their centre is

$$\bar{\mathbf{g}} = \frac{1}{\mu} \sum_{i=1}^{\mu} \mathbf{g}_i. \tag{2.5}$$

Each new candidate is sampled by adding isotropic Gaussian noise,

$$\mathbf{g}' = \bar{\mathbf{g}} + \sigma \boldsymbol{\epsilon}, \qquad \boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, I), \tag{2.6}$$

where σ controls the mutation scale. The best genome encountered so far is retained unchanged through elitism. Unlike a GA with crossover, ES1 does not preserve and recombine discrete parent genomes. It summarizes the selected set by its mean and explores a neighbourhood around that point.

3 Original Study: Methodology and Results

This chapter presents the completed original study as three linked experiments. Section 3.1 defines the implementation, controllers, tasks, and protocol that all three experiments share. Section 3.2 compares held-out performance, Section 3.3 tests robustness to unseen initial weights, and Section 3.4 examines whether lifetime updates create reusable weights.

3.1 Shared Experimental Framework

3.1.1 Implementation and Reproducibility

Python implements the experiments, Gymnasium exposes the EvoGym environments, and PyTorch networks control the robots. A YAML configuration records the task, morphology, controller, network, evolutionary budget, evaluation protocol, runtime settings, and training seed. Before training, the implementation resolves symbolic network dimensions against the instantiated environment.

The implementation uses separate run seeds. Optimizer sampling, environment resets, and NcHL weight initialization therefore do not share one global generator. Before each reset, the implementation also reseeds the global Python generator for EvoGym tasks that use it internally. BidirectionalWalker, for example, selects its target direction during reset.

Each completed run stores a resolved configuration, generation-level metrics and the best genome.

3.1.2 Controller Implementations

Both conditions use the same bias-free feed-forward topology. The instantiated task and morphology determine the input width. The network contains one hidden layer with the same width as the observation vector and one output for each actuator voxel. Every layer applies a hyperbolic tangent activation:

$$\mathbf{a}^{(\ell+1)} = \tanh\left(W^{(\ell)} \mathbf{a}^{(\ell)}\right). \tag{3.1}$$

The domain of the output is $[-1, 1]$. A piecewise-linear transformation maps zero to the neutral actuator target 1.0, negative values towards the lower action bound, and positive values towards the upper bound. This transformation preserves a meaningful neutral command when the bounds are asymmetric.

For raw network output $a_i \in [-1, 1]$, lower bound l_i, and upper bound u_i, the physical actuator command is

$$\widetilde{a}_i = \begin{cases} 1 + (1 - l_i)a_i, & -1 \le a_i < 0, \\ 1 + (u_i - 1)a_i, & 0 \le a_i \le 1. \end{cases} \tag{3.2}$$

Thus $a_i = -1$, 0, and 1 map exactly to l_i, 1.0, and u_i, respectively.

The static condition evolves every matrix entry and loads the resulting values at the start of an episode. These weights remain fixed. The NcHL condition evolves per neuron parameters and samples concrete weights independently from a configured uniform interval at every episode reset. After each forward pass, the controller immediately applies the update with the pre- and postsynaptic activations cached during that pass.

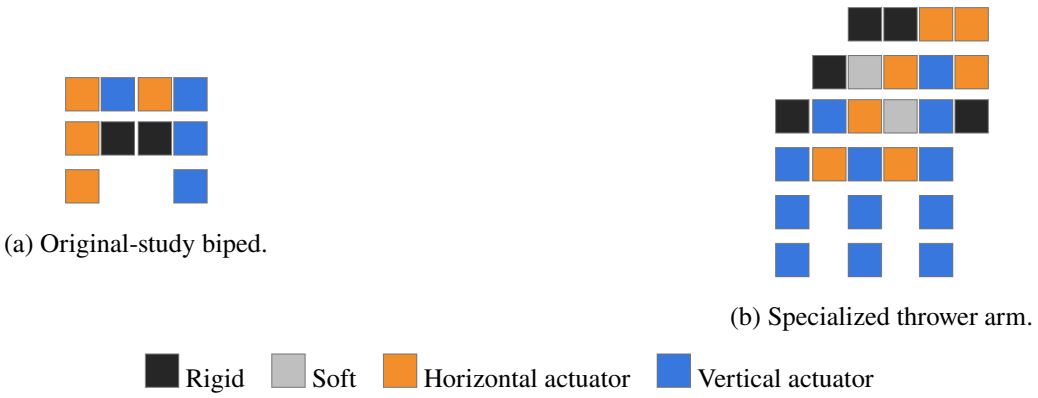
The representations differ in genome length and evaluation budget, therefore the test is not a fixed compute comparison. Table 3.2 reports both shared and controller-specific settings.

3.1.3 Tasks, Morphologies, and Evolution

The original suite contains Walker-v0, Carrier-v0, Jumper-v0, and Thrower-v0. Every task uses the biped body; Thrower also uses a specialized arm. The two Thrower bodies differ in size and actuator count, so their comparison remains descriptive rather than controlled and is simply a study into the effect of morphology.

Table 3.1: Original task–morphology conditions. The names follow the archived run configurations.

Task	Morphology	Horizon	Principal Behaviour
Walker-v0	biped	500	Forward locomotion on flat terrain
Carrier-v0	biped	500	Object transport during locomotion
Jumper-v0	biped	500	Vertical impulse generation
Thrower-v0	biped	300	Object acceleration and release
Thrower-v0	thrower_arm	300	Throwing with specialized geometry



(a) Original-study biped.

(b) Specialized thrower arm.

Figure 3.1: Morphologies used in the original study.

Each combination of task, morphology, controller, and seed forms an independent optimization run. Both conditions use a population of 40 for 500 generations, select ten parents, apply Gaussian perturbation with $\sigma = 0.35$, and retain the best genome observed so far. Static candidate fitness uses one episode. NcHL candidate fitness uses the mean of three episodes with distinct initialization and environment seed pairs in an attempt to generalize the rule across initializations.

The evidence comprises 100 evolutionary runs, 1,000 held-out episodes, 300 weight traces, and 750 frozen-controller rollouts. The candidate evaluations add four million nested training episodes. All principal analyses use a trained run as the independent unit.

Table 3.2: Shared and controller-specific settings in the original study.

Item	Value
Optimizer	ES1 with parent-mean Gaussian sampling and elitism
Population and generations	40 candidates and 500 generations
Parent count and mutation scale	$\mu = 10$ and $\sigma = 0.35$
Initial evolutionary population	Uniform samples from $[-1, 1]$
Candidate budget	One episode for Static-ES; three for NcHL-ES
Initial NcHL runtime weights	Independent uniform samples from $[-0.1, 0.1]$
NcHL learning rate	$\eta = 0.1$
Held-out evaluation	Ten episodes for each selected genome
Training seeds	0–9

3.2 RQ1: Evolutionary Training Dynamics

3.2.1 Experimental Design

This experiment uses the generation-level metrics from the ten independent evolutionary runs for each controller and task–morphology condition. At every generation, the analysis records the best fitness in the current population, the best fitness observed so far, and the population median. The curves summarize the across-run mean best-so-far fitness with a bootstrap interval and show the mean population median as a dashed line.

To determine whether the numerical fitness corresponds to recognizable task behaviour, an inspection of the footage of the best saved controller from each seed was performed. This visual assessment is qualitative: it helps distinguish genuine task completion from reward improvement that does not produce the intended behaviour.

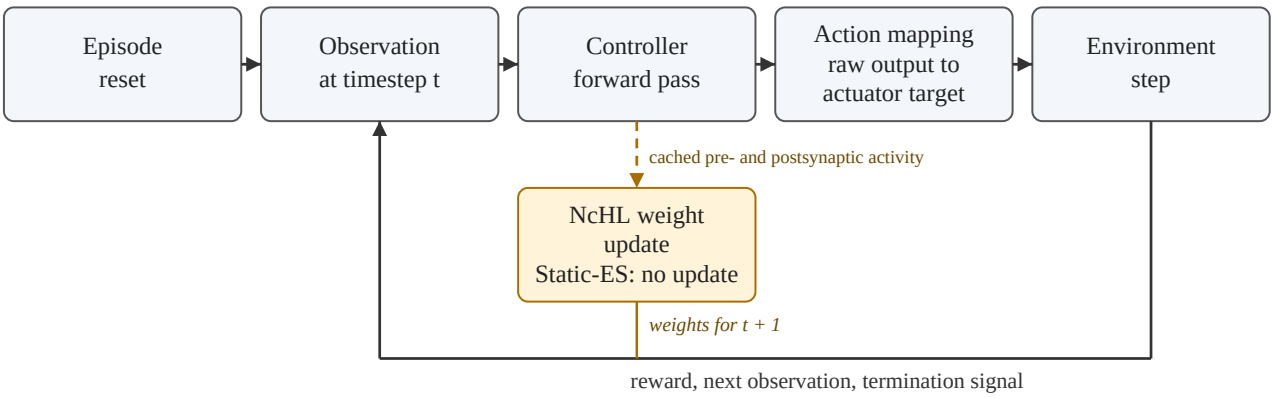


Figure 3.2: Within-episode control loop used to evaluate one candidate. The loop repeats until termination; accumulated reward becomes candidate fitness. The lower branch shows the additional online update performed only by NcHL-ES.

3.2.2 Results

The evolutionary curves show improvement for both representations. NcHL-ES reaches a mean selected training fitness 4.2% above Static-ES on Walker and 55.1% above it on Jumper. Static-ES finishes higher on Carrier and both Thrower morphologies. Figure 3.3 also shows substantial variation across independent runs, particularly for NcHL-ES.

The footage provides important context for these curves. On Walker, most seeds of both NcHL-ES and Static-ES produce natural-looking locomotion. The biped and specialized-arm Thrower controllers can load the box and then release it with a recognizable throwing movement. These successful conditions show that NcHL can support both cyclical locomotion and non-cyclical, phase-structured behaviour.

Carrier and Jumper do not provide equally strong evidence. The Carrier controllers drop the box almost immediately, whereas the Jumper controllers consistently stretch upwards and then freeze; only the best examples

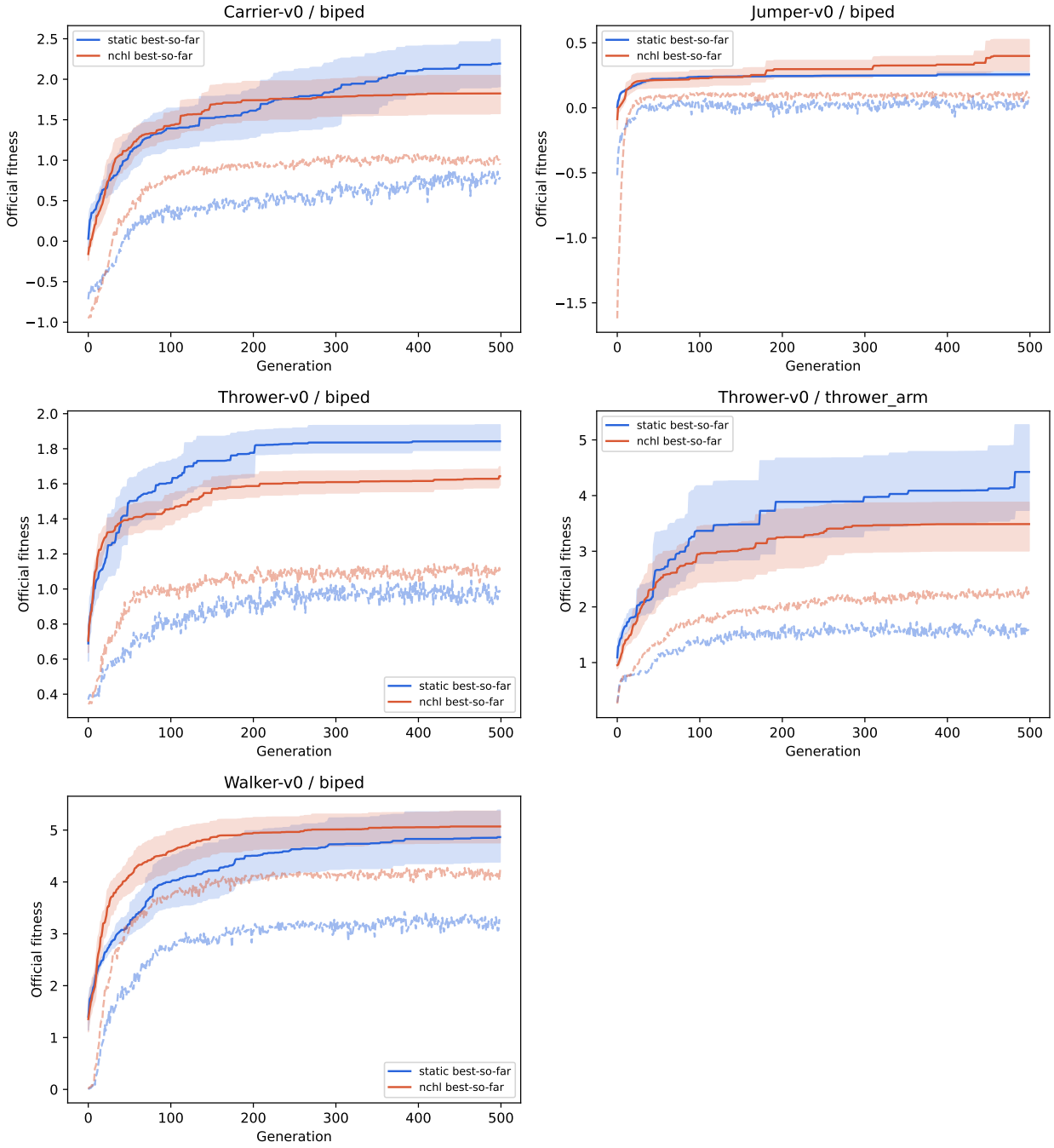


Figure 3.3: Evolutionary training dynamics across ten runs per controller and condition. Solid lines show mean best-so-far fitness, shaded regions show bootstrap intervals, and dashed lines show the mean population median.

produce a small jump. The tested morphologies and training setup therefore fail to solve these two tasks in behavioural terms. Their fitness curves still describe optimization under the task reward, but differences between NcHL-ES and Static-ES in these conditions should not be treated as strong evidence about controller capability. In particular, the numerical NcHL advantage on Jumper does not correspond to a convincing solution of the task.

RQ1 Summary. Both representations improve during evolution, but the fitness curves do not always indicate successful task behaviour. Walker and both Thrower morphologies produce meaningful solutions, demonstrating that NcHL can support both cyclical and phase-structured control. Carrier and Jumper remain behaviourally unsolved with the tested morphologies, so their relative fitness values provide weak evidence for comparing the controllers.

3.3 RQ2: Robustness to Unseen Initial Weights

3.3.1 Experimental Design

An NcHL genome encodes a rule rather than one final weight matrix. This experiment applies every selected rule to initial weights generated from seeds outside its training stream. The ten held-out episodes for each run vary both the environment seed and controller seed. Because the original nominal environments behave deterministically after their declared reset, the sampled runtime weights dominate the additional within-run dispersion.

We compare the selected training fitness for each run—the maximum three-episode candidate estimate observed during evolution—with the mean over ten held-out initializations. The difference addresses practical generalization but can combine three mechanisms: specialization to fixed training pairs, optimism from selecting the maximum noisy estimate, and an inability to recover from arbitrary W_0. This design establishes the gap but cannot separate those causes.

3.3.2 Results

All 50 selected NcHL solutions achieve a lower mean across ten unseen initial weights than their selected three-initialization training estimate. Table 3.3 reports the condition means. Retained performance divides the held-out mean by the selected training mean.

Table 3.3: Generalization of selected NcHL rules to unseen runtime-weight initializations. Within-run SD summarizes ten nested episodes.

Condition	Training	Held-out	Gap	Retained	Within-run SD	Negative Episodes
Walker, biped	5.069	4.395	−0.674	86.7%	0.691	1/100
Carrier, biped	1.825	1.191	−0.634	65.3%	0.464	3/100
Jumper, biped	0.399	0.024	−0.375	6.0%	0.369	35/100
Thrower, biped	1.643	1.215	−0.429	73.9%	0.289	2/100
Thrower, arm	3.488	2.233	−1.255	64.0%	0.613	8/100

Jumper provides the strongest failure case. Its held-out mean retains only 6.0% of the selected training mean, four of ten run means become negative, and 35 of 100 nested episodes produce negative return. Walker retains 86.7%, whereas Carrier and the two Thrower conditions retain approximately 64–74%. Static-ES reproduces its selected weights during evaluation and therefore has no corresponding variation from weight initialization.

Evolution evaluates three fixed initialization pairs per run and selects the best noisy candidate encountered across 500 generations. Lower held-out performance can therefore reflect specialization, selection optimism, or insufficient recovery from arbitrary starting weights. The data establish the practical result that new runtime weights remain consequential in every condition.

RQ2 Summary. The results indicate that weight initialization influences the behaviour produced by the evolved learning rules. Nevertheless, the Walker and Thrower controllers retain meaningful task behaviour across initializations, suggesting that the rules capture useful structure rather than relying on a single favourable

initialization. The poor performance on Carrier and Jumper is less informative about robustness because these tasks were not successfully learned under the nominal evaluation either.

3.4 RQ3: Do Lifetime Updates Create Useful Weights?

3.4.1 Freeze-and-Replay Design

This experiment reuses the best NcHL genome from every original run and performs no further evolutionary search. Three independently seeded plastic source episodes generate concrete weight matrices at five normalized lifetime positions:

$$W_0, \quad W_{25}, \quad W_{50}, \quad W_{75}, \quad W_{\text{final}}. \tag{3.3}$$

The trace records W_0 before the first action. It records W_{final} after the update that follows the last source action, so this final matrix might never generate an action in the source trajectory.

The replay procedure loads each matrix into a static network and starts a fresh episode with the environment seed of the corresponding source trace. It disables plasticity throughout replay. This intervention tests whether an acquired matrix remains useful from the normal initial state; it does not continue from the physical state that produced the matrix.

For every snapshot, we first average the three traces within each evolutionary run and then summarize the runs. The principal quantities are the frozen-fitness change from W_0, the lifetime stage with the best frozen fitness, and the difference between the fully plastic source return and each frozen controller. Improvement over W_0 supports lifetime improvement.

3.4.2 Results

The experiment starts from a deliberately weak baseline. Before adaptation, random W_0 networks produce near-zero Walker fitness, negative Carrier and Jumper fitness, and low Thrower fitness. Later snapshots acquire useful fresh-start behaviour in several conditions, but the pattern is neither uniform nor monotonic (Table 3.4).

Table 3.4: Fresh-start frozen fitness across lifetime snapshots. Values first average three traces within each run and then average ten run means. "Improved" counts runs for which W_{final} exceeds W_0.

Condition	Plastic	W_0	W_{25}	W_{50}	W_{75}	W_{final}	Final Change	Improved
Walker, biped	4.557	0.000	2.263	2.242	2.030	2.381	+2.381	8/10
Carrier, biped	1.276	−0.490	−0.083	−0.019	−0.017	0.004	+0.493	7/10
Jumper, biped	0.167	−0.168	−2.133	−2.912	−2.705	−2.093	−1.925	0/10
Thrower, biped	1.248	0.338	0.338	0.346	0.482	0.498	+0.160	8/10
Thrower, arm	2.252	0.243	1.087	1.037	1.172	1.369	+1.125	9/10

Plastic control outperforms the matched W_0 frozen ablation in 49 of 50 runs. It also exceeds W_{final} in 47 runs and the post-hoc best frozen snapshot in 46.

Only five of the 50 frozen trajectories improve monotonically across all five snapshots. The learning rule therefore does not simply approach one final static optimum. A late matrix can perform poorly from the initial physical state while remaining useful during the phase that produced it.

RQ3 Summary. Online updates contribute to performance, but they do not generally converge towards one progressively better fixed controller. The Walker result however is interesting: its highest frozen fitness occurs at W_{final}, suggesting that tasks with periodic optimal movement may show an upward fitness trend as the repeated

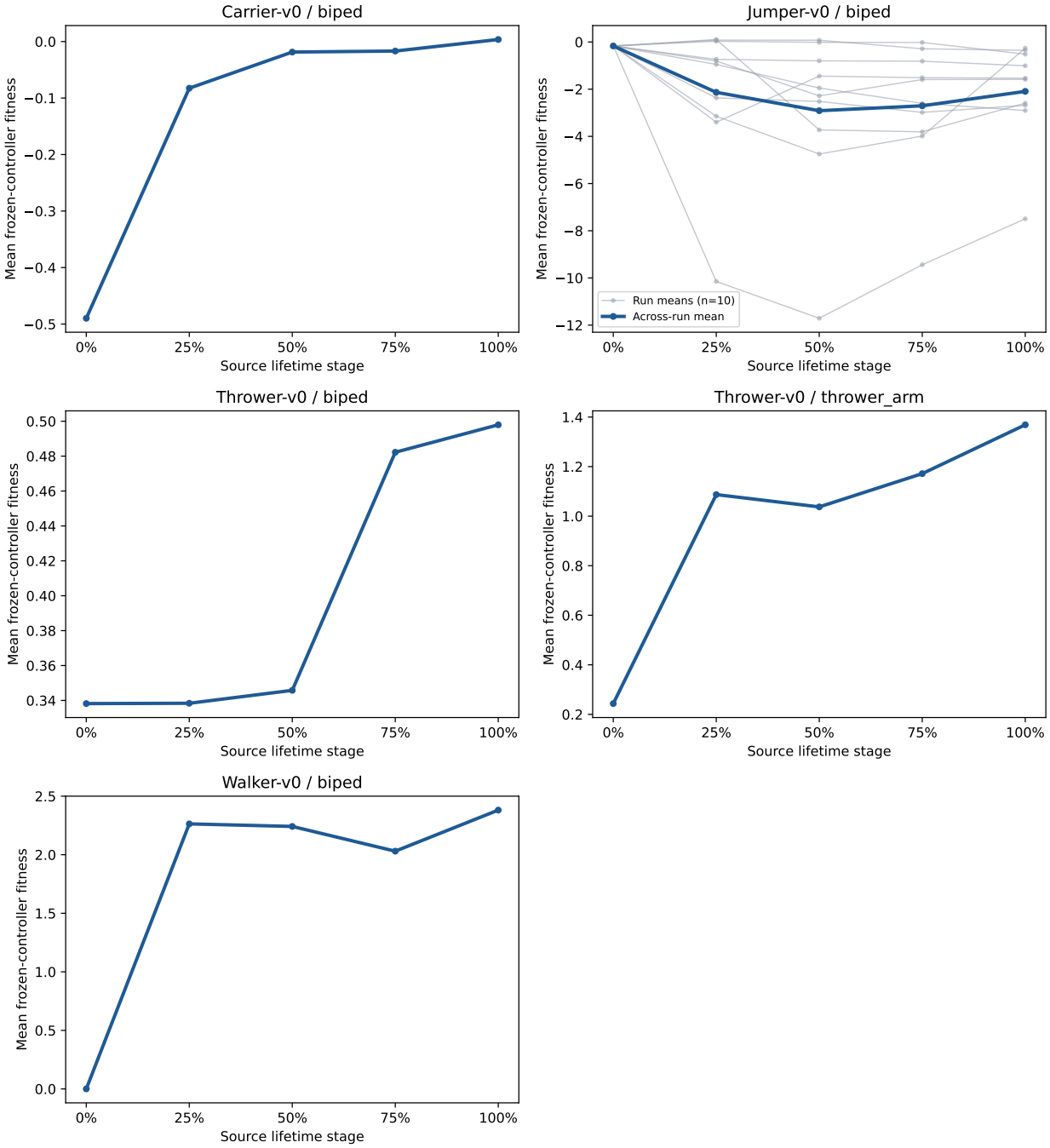


Figure 3.4: Across-run mean fitness of frozen weight snapshots evaluated from a fresh task start. The Jumper panel retains the ten run means in grey to illustrate the large between-seed variation; the blue line gives the across-run mean in every panel.

control loop is progressively optimized.

4 Extended Study: Task and Morphology Dependence

4.1 Motivation and Research Question

The original study establishes that NcHL can evolve useful behaviour, but its small task set cannot show whether performance follows a general task property. The extended study addresses the limitations by comparing NcHL-ES and Static-ES on twelve tasks and three controlled bodies.

This experiment answers RQ4: *which task characteristics are associated with good NcHL performance, and are those patterns independent of morphology?* The chapter first reports the predefined group comparison and then uses individual tasks to interpret the physical and temporal demands behind it.

4.2 Experimental Design

4.2.1 Task Selection and Predefined Groups

The twelve environments were assigned to four groups before their results were inspected (Table 4.1). The groups describe a dominant control demand.

Table 4.1: Predefined task groups in the extended experiment.

Group	Tasks	Dominant characteristic
Continuous-support locomotion	Walker; BridgeWalker; BidirectionalWalker	Repeated locomotion on rigid or deformable support, including changing goals
Terrain negotiation	DownStepper; ObstacleTraverser; CaveCrawler	Contact adaptation, terrain sensing, step transitions, and clearance control
Object interaction	Pusher; Carrier; Thrower	Control while mechanically coupled to an external object
Whole-body dynamic control	Flipper; Jumper; Balancer	Rotation, impulse generation, or postural regulation without sustained transport

4.2.2 Control Demands of the Individual Tasks

The group labels conceal differences that may matter to an activity-dependent controller. Based on the objectives and mechanics of the EvoGym environments, Table 4.2 gives the author's classification of the principal temporal and physical demand of every task [1]. In particular, a repetitive task can retain one objective, as in Walker, or require repeated goal changes, as in BidirectionalWalker. Likewise, object interaction can require sustained contact or one precisely timed release.

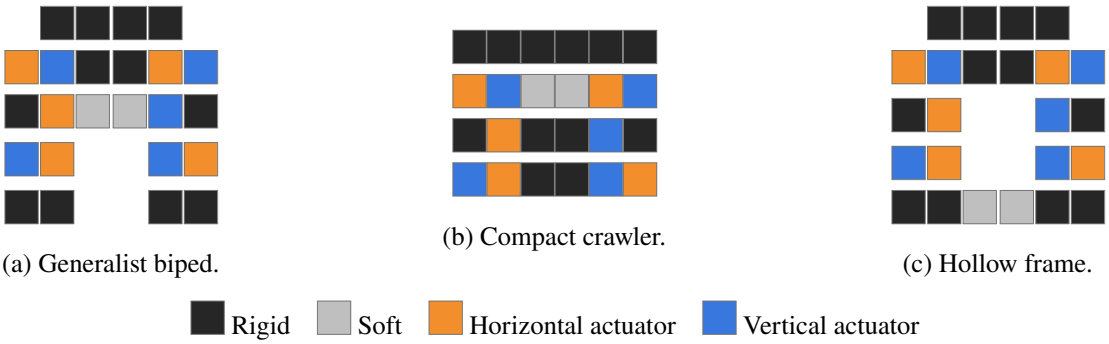
4.2.3 Controlled Morphologies

Every task is evaluated with a generalist biped, compact crawler, and hollow frame. Each body contains 24 occupied voxels: 12 rigid, two soft, five horizontal actuators, and five vertical actuators. The action dimension is therefore ten in every condition. Geometry and mass distribution change, while occupied volume, material counts, and actuator budget remain fixed.

The biped separates support into leg-like columns, the crawler forms a low and nearly rectangular contact structure, and the frame redistributes the same material around a central opening.

Table 4.2: Task characteristics used to interpret the controller comparison.

Task	Temporal structure	Principal physical or control demand
Walker	Repeated, fixed direction	Locomotion on rigid continuous support
BridgeWalker	Repeated, fixed direction	Locomotion coupled to deformable support
BidirectionalWalker	Sequential goals	Direction changes and context-dependent action
DownStepper	Repeated with transitions	Descending contacts while remaining upright
ObstacleTraverser	State-dependent	Terrain sensing, clearance, and balance
CaveCrawler	Repeated under constraint	Simultaneous floor and ceiling clearance
Pusher	Sustained progression	Persistent contact with a movable object
Carrier	Sustained progression	Transport while retaining the load
Thrower	Phase structured	Load, accelerate, and release
Flipper	Repeated, one direction	Continued whole-body rotation
Jumper	Ballistic	Compression, launch, and height retention
Balancer	Stabilizing	Approach and maintain a target state



(a) Generalist biped. (b) Compact crawler. (c) Hollow frame.

Figure 4.1: Controlled morphologies.

4.2.4 Experimental Scale and Completeness

The design contains $12 \times 3 \times 2 \times 10 = 720$ independent optimization runs. Every run completed 500 generations with a population of 40 and mutation scale 0.35. Static candidates use one episode, while NcHL candidates again average three runtime-weight initializations.

4.3 Analysis Strategy

The task-level curves first average the three morphology scores within each seed and then summarize the ten independent seeds. Their outcome is best-so-far fitness during evolution. Final comparisons use the selected best-so-far score at generation 500. Footage of the evolved controllers supplies a separate behavioural check: a higher task reward is not treated as evidence of controller capability when the robot is not able to perform the intended task recognizably. Numerical results from such conditions remain reported, but receive little weight.

Raw return is meaningful within a task but cannot be pooled across tasks. For task t, morphology m, and seed s, the controller difference is therefore standardized by the sample standard deviation of the 20 final scores in that task–morphology stratum:

$$z_{tms} = \frac{F_{tms}^{NcHL} - F_{tms}^{\text{static}}}{s_{tm}}. \tag{4.1}$$

Positive values favour NcHL-ES and negative values favour Static-ES. Group effects average the nine standardized values contributed by three tasks and three morphologies within each seed. Means are accompanied by non-parametric bootstrap intervals over the ten seed-level values. Raw within-task differences are retained where standardization could obscure practical magnitude.

4.4 Results Across Tasks and Morphologies

4.4.1 Training Dynamics and Task-Group Results

Both controller representations improve numerically on every task, but they do not reach the same training ceiling (Figure 4.2). Walker, BridgeWalker, Pusher, Jumper, and CaveCrawler show rapid early gains followed

by a plateau. Static-ES continues to separate on BidirectionalWalker, DownStepper, ObstacleTraverser, Carrier, Thrower, and Balancer.

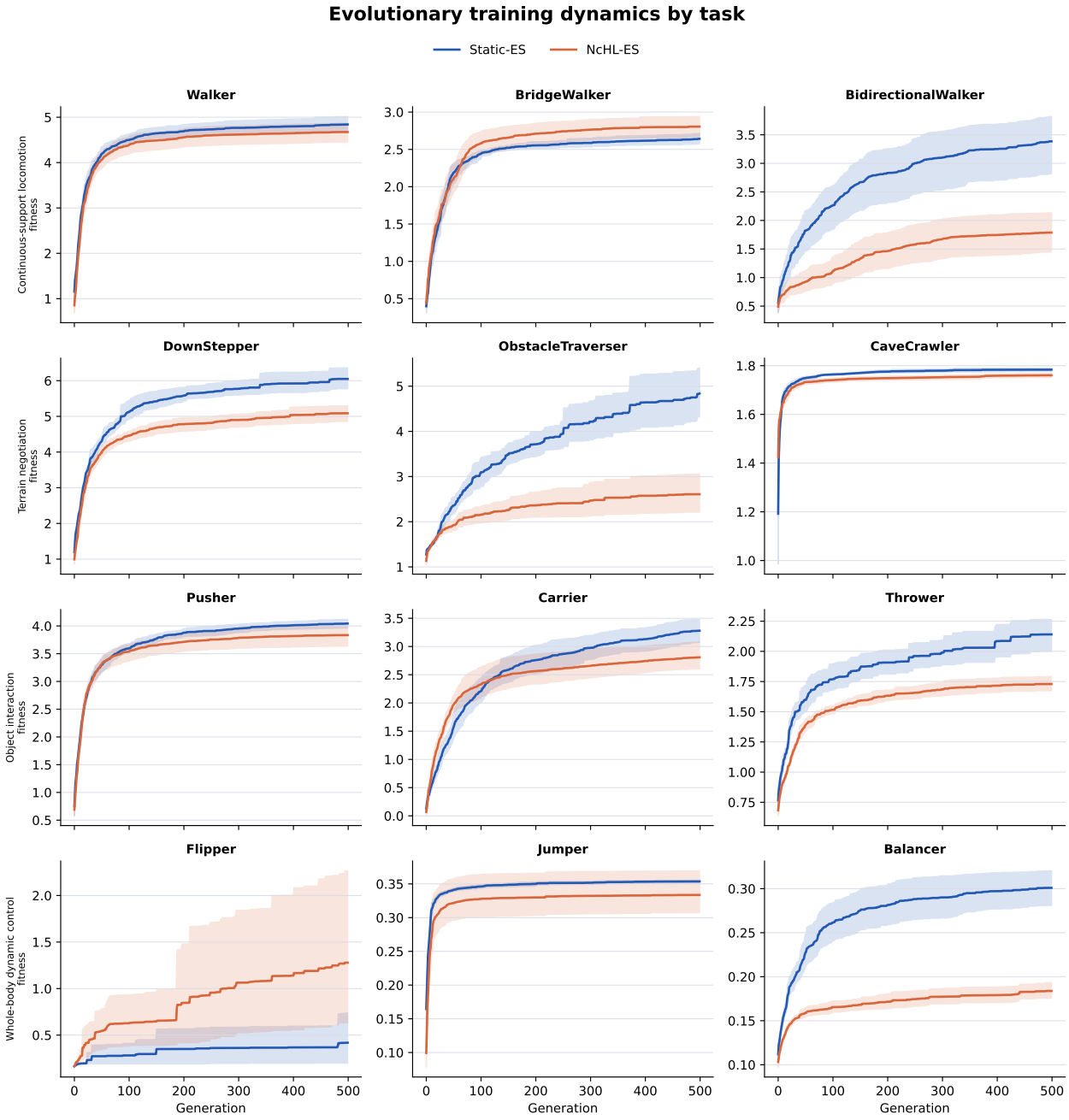


Figure 4.2: Best-so-far evolutionary fitness for the twelve extended tasks. Each task curve first averages the three morphologies within a seed and then summarizes ten seeds; shaded bands are 95% bootstrap intervals. Panels retain their task-specific reward scales.

None of the four predefined groups produce an aggregate NcHL advantage (Table 4.3). Terrain negotiation is most negative. Whole-body dynamic control is least negative numerically because the Flipper score partly offsets Jumper and Balancer. The footage prevents this offset from being interpreted as a positive behavioural result: none of the three whole-body tasks is learned recognizably.

Table 4.4 summarizes the behavioural evidence used to qualify these scores.

4.4.2 Task-by-Morphology Results

The aggregate group results conceal a strong interaction with body geometry (Figure 4.3). Only eight of the 36 numerical cells favour NcHL, and five of those use the compact crawler. Flipper is the only task with a positive score effect for every body, but its footage shows no successful solution. Conversely, BidirectionalWalker,

Table 4.3: Mean standardized final training effect by predefined task group. Positive values favour NcHL-ES.

Group	Mean effect	95% interval	Positive seed effects
Continuous-support locomotion	−0.58	[−0.70, −0.47]	0/10
Terrain negotiation	−1.15	[−1.33, −0.91]	0/10
Object interaction	−0.67	[−0.89, −0.40]	1/10
Whole-body dynamic control	−0.44	[−0.77, −0.07]	1/10

Table 4.4: Qualitative assessment of evolved-controller footage.

Group	Observed behaviour	Interpretive weight
Continuous-support locomotion	Controllers produce recognizable goal-directed locomotion	Informative
Terrain negotiation	DownStepper is adequate; Static-ES performs acceptably on ObstacleTraverser while NcHL does not; CaveCrawler is not learned	Task-specific; CaveCrawler carries little weight
Object interaction	Recognizable object-directed behaviour emerges	Informative
Whole-body dynamic control	Flipper, Jumper, and Balancer are not learned recognizably	Low; fitness differences do not establish task competence

DownStepper, ObstacleTraverser, CaveCrawler, Thrower, and Balancer are negative for all three morphologies.

Table 4.5 summarizes the task averages and the morphology pattern behind them. BridgeWalker illustrates why both levels are required. Its raw task-average score is 6.2% higher for NcHL, but the positive result comes entirely from the compact crawler: its standardized effect is +1.86, compared with −1.59 for the biped and −0.95 for the frame. Seeing as continuous-support locomotion is at least partially learned in the footage, this interaction is behaviourally informative.

Table 4.5: Task-level final training comparison. The raw percentage is meaningful only within its row. The standardized effect averages the three morphology-specific effects.

Task	Raw difference	Mean effect	Morphology pattern
Flipper	+205.7%	+1.01	Positive for all three bodies
BridgeWalker	+6.2%	−0.23	Strong crawler advantage; other bodies negative
Walker	−3.4%	−0.24	Crawler slightly positive; other bodies negative
Pusher	−5.2%	−0.33	Crawler positive; other bodies negative
Carrier	−14.4%	−0.37	Crawler positive; other bodies negative
Jumper	−5.7%	−0.74	Small, inconsistent frame advantage only
DownStepper	−15.9%	−0.86	Crawler closest to neutral
CaveCrawler	−1.3%	−1.13	Small but consistent raw disadvantage
BidirectionalWalker	−47.2%	−1.28	Negative for all three bodies
Thrower	−19.2%	−1.31	Negative for all three bodies
ObstacleTraverser	−46.1%	−1.46	Negative for all three bodies
Balancer	−38.9%	−1.58	Negative for all three bodies

4.5 Task Characteristics and Answer to RQ4

4.5.1 Favourable and Unfavourable Task Demands

Continuous-support locomotion. This group provides the clearest positive behavioural evidence in the extended suite. The controllers learn the tasks and produce recognizable locomotion. The aggregate group effect still favours Static-ES, so continuous support is not sufficient for an NcHL advantage. The more specific pattern is an interaction with body geometry: NcHL is positive for the compact crawler on Walker and especially BridgeWalker, but negative for the biped and frame. The evidence therefore supports compatibility between repeated locomotion and a low, stable body rather than a general claim about cyclical movement.

Object interaction. The object tasks are also learned recognizably, which makes their score differences relevant. Pusher and Carrier favour NcHL with the compact crawler, whereas the other bodies and all Thrower

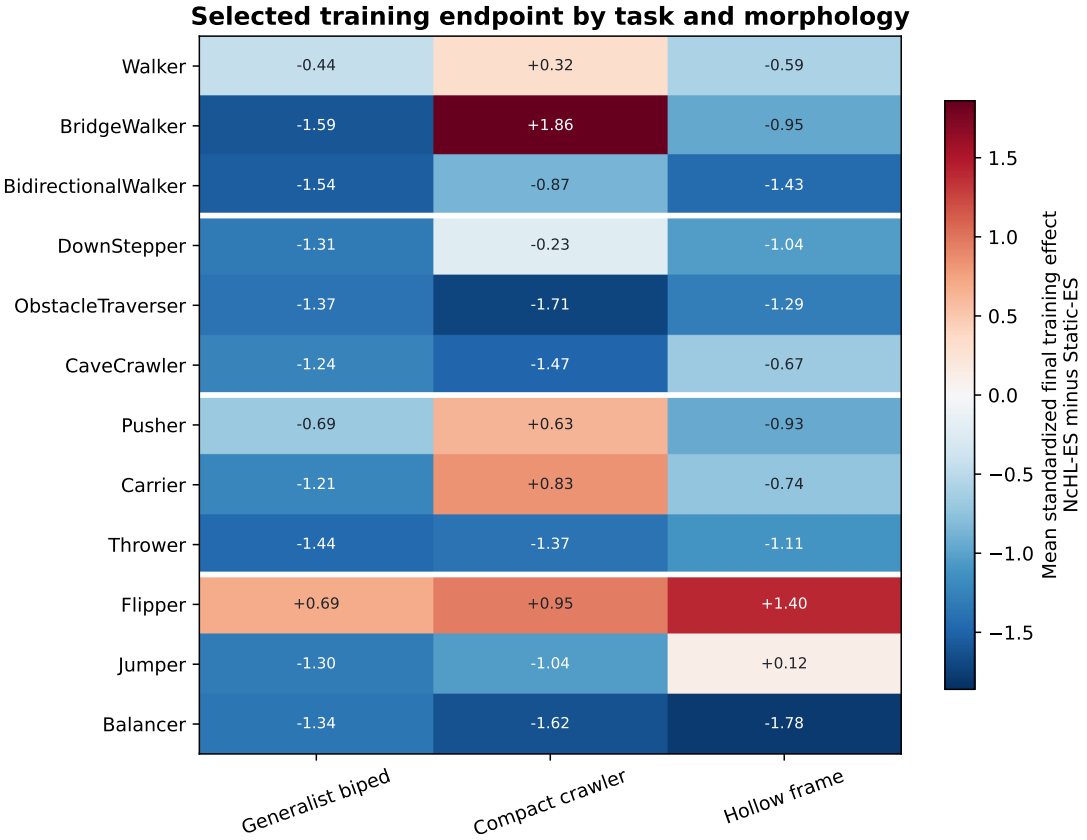


Figure 4.3: Mean standardized final training effect for every task–morphology condition. Positive values favour NcHL-ES; negative values favour Static-ES. Horizontal separators indicate the four predefined task groups.

comparisons favour Static-ES. The negative Thrower result may reflect its more specific loading and release demands.

Terrain negotiation. The group combines informative and uninformative conditions. DownStepper produces acceptable behaviour and can be interpreted as a genuine controller comparison; its scores favour Static-ES overall, with the crawler closest to neutral. On ObstacleTraverser, Static-ES produces acceptable traversal while NcHL fails to perform the task, making this the clearest behavioural disadvantage for NcHL in the group. CaveCrawler is learned by neither representation. Its small but consistent numerical difference therefore says little about competent terrain control and should not influence the task-characteristic conclusion strongly.

Whole-body dynamic control. None of the Flipper, Jumper, or Balancer controllers learns the intended task recognizably. The apparently strong Flipper score is therefore not evidence that repeated rotation favours NcHL. Similarly, the negative Jumper and Balancer effects do not demonstrate that NcHL is intrinsically unsuitable for impulse generation or stabilization: the experiment did not first establish successful behaviour with either representation. This group mainly reveals a limitation of the tested morphologies, controller architecture, or training setup.

4.5.2 Morphology Dependence

The compact crawler is the most favourable body numerically, with a mean effect of -0.31 across tasks, compared with -0.75 for the hollow frame and -1.07 for the biped. More importantly, its positive results on Walker, BridgeWalker, Pusher, and Carrier occur in the two groups where the footage shows recognizable task learning. For continuous-support locomotion, the crawler effect is $+0.44$ with a 95% interval of $[+0.27, +0.62]$, and every seed-level group average is positive. Its low rectangular geometry and broad contact area may permit useful repeated deformation without the same balance and foot-placement demands as the biped or frame.

The positive Flipper cell is excluded from this behavioural argument because the task is not learned. Even

after that exclusion, the remaining informative positive cells are concentrated in the crawler. The evidence therefore indicates a task–body interaction rather than a task characteristic that operates independently of morphology. The three selected bodies cannot identify whether contact area, height, mass distribution, or another geometric property is responsible.

4.5.3 RQ4 Summary

The experiment does not identify a broad task characteristic that makes NcHL consistently successful. The most credible positive evidence occurs in partially learned continuous-support and object-interaction tasks when they are paired with the compact crawler.

The observed pattern is therefore strong morphology-dependency. NcHL may be compatible with repeated locomotion or sustained contact when the body supplies a stable mechanical foundation, but the current evidence does not establish an advantage for cyclicality, rotation, impulse generation, or stabilization as general task properties.

5 Conclusion

5.1 Conclusions and Limitations

NcHL can produce competitive behaviour relative to Static-ES on most tasks that the tested controllers learn recognizably. Both representations produce natural locomotion and structured throwing in the original study; the extension produces partial continuous-support locomotion and object interaction, as well as adequate DownStepper behaviour. Static-ES remains more reliable and is clearly stronger in some conditions.

NcHL's online updates are also meaningful: plastic control usually improves on the random initial network, and Walker acquires reusable weights. However, the rules remain sensitive to initialization and do not generally converge towards one progressively better fixed controller.

No general task characteristic reliably predicts good NcHL performance. None of the four predefined groups produces an aggregate advantage, while CaveCrawler and all three whole-body dynamic tasks remain too poorly learned for their scores to establish controller capability. Morphology dependence is much clearer: most informative positive cells use the compact crawler in partially learned locomotion and object-interaction tasks. The evidence therefore supports task–body compatibility, not a morphology-independent distinction such as cyclical versus non-cyclical control.

The conclusions are limited to simulation, twelve selected tasks, and three hand-designed bodies. The methods differ in genome size and candidate episode budget, so the comparison is not compute-matched. Results may also depend on the bias-free tanh architecture, ES1, and unbounded update. Fresh-start freezing may underestimate phase-specific weight value. Transfer to physical robots, modelling error, and damage remain untested.

5.2 Future Work

Two interesting approaches could investigate the observed dependence on morphology. The first would systematically compare a larger set of morphologies with controlled geometric characteristics, such as different bipedal layouts or asymmetric bodies. The second would co-evolve the controller and morphology, allowing body design and control strategy to adapt jointly.

A systematic comparison across many morphologies could identify which body characteristics influence NcHL performance. Properties such as symmetry, contact area, limb arrangement, and actuator placement could be varied while keeping material and actuator budgets constant. This would help explain which specific features cause the observed morphology dependence.

Controller–morphology co-evolution would test whether evolving the body design alongside the controller can free NcHL from the task-specific limitations imposed by fixed morphologies.

Bibliography

[1] Jagdeep Bhatia, Holly Jackson, Yunsheng Tian, Jie Xu, and Wojciech Matusik. Evolution gym: A large-scale benchmark for evolving soft robots. In *Advances in Neural Information Processing Systems 34: Proceedings of the 35th Conference on Neural Information Processing Systems*, pages 2201–2214, Virtual conference, 6–14 December 2021.

[2] A. E. Eiben and J. E. Smith. *Introduction to Evolutionary Computing*. Springer, second edition, 2015.

[3] Andrea Ferigo, Elia Cunegatti, and Giovanni Iacca. Neuron-centric hebbian learning. In *Proceedings of the Genetic and Evolutionary Computation Conference (GECCO '24)*, pages 617–625, Melbourne, VIC, Australia, 14–18 July 2024. Association for Computing Machinery.

[4] Andrea Ferigo, Giovanni Iacca, Eric Medvet, and Federico Pigozzi. Evolving hebbian learning rules in voxel-based soft robots. *IEEE Transactions on Cognitive and Developmental Systems*, 15(3):1536–1546, 2023.

[5] Daniela Rus and Michael T. Tolley. Design, fabrication and control of soft robots. *Nature*, 521(7553):467–475, 2015.

[6] Andrea Soltoggio, Kenneth O. Stanley, and Sebastian Risi. Born to learn: The inspiration, progress, and future of evolved plastic artificial neural networks. *Neural Networks*, 108:48–67, December 2018.